

Competitive Programming

Section 60 Editorial: Final Capstone

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. Capstone Subset	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
3	B. Capstone CRT	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
4	C. Capstone Recurrence	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. Capstone Subset Optimization. Split the items into two halves, enumerate all subset sums of each half, and sort the right-half sums. For every left sum, binary-search the best right partner that keeps the total under the limit. This is the meet-in-the-middle proof from Section 47: every full subset has exactly one left subset and one right subset.

B. Capstone Congruences. Merge congruences one at a time with CRT. At each step, the current pair (a, m) represents all numbers satisfying the processed constraints. Merging with (b, n) either detects incompatible residues modulo $\gcd(m, n)$ or produces a new residue modulo $\text{lcm}(m, n)$ representing the intersection. This invariant makes the final answer immediate.

C. Capstone Recurrence. Use fast doubling to evaluate the recurrence in logarithmic time. The state pair stores adjacent Fibonacci values, the doubling formula advances from k to $2k$, and the optional step advances to $2k+1$. Processing all bits of n therefore computes the requested value exactly.

Common pitfalls. The final capstone intentionally mixes techniques. Write the invariant for each module separately, test it locally, then combine the outputs only after each module is correct on its own.

The capstone revisits three patterns that should now be automatic.

- A uses meet-in-the-middle for maximum subset sum under a limit.
- B merges congruences by generalized CRT and rejects incompatible residues.
- C answers recurrence queries with fast Fibonacci doubling.

The final proof style is concise but complete: define the compressed object, prove it preserves the original choices, and show every query reads the desired value from that object.

2 A. Capstone Subset

2.1 Restatement

Meet-in-the-middle: maximum subset sum not exceeding X .

2.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

2.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

2.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<long long>sums(vector<long long>a){vector<long long>r{0};for(lon
```

2.6 Full Python solution

```
import sys,bisect
def sums(a):
    res=[0]
    for x in a: res += [y+x for y in res]
    return res
def main():
```

```

data=list(map(int,sys.stdin.buffer.read().split()))
if not data: return
n,x=data[0],data[1]; a=data[2:]; left=sums(a[:n//2]); right=sorted(sums(a[n//2:])); best=0
for s in left:
    i=bisect.bisect_right(right,x-s)-1
    if i>=0: best=max(best,s+right[i])
print(best)
if __name__=="__main__": main()

```

3 B. Capstone CRT

3.1 Restatement

Merge pairs of congruences or report impossibility.

3.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

3.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

3.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; long long eg(long long a,long long b,long long&x,long long&y){if(!b){x=
```

3.6 Full Python solution

```
import sys,math
def eg(a,b):
    if b==0: return (abs(a),1 if a>=0 else -1,0)
    g,x,y=eg(b,a%b); return g,y,x-(a//b)*y
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
```

```

if not data: return
t=data[0]; idx=1; out=[]
for _ in range(t):
    a,m,b,n=data[idx],data[idx+1],data[idx+2],data[idx+3]; idx+=4; g,x,y=eg(m,n)
    if (b-a)%g: out.append("NO")
    else:
        l=m//g*n; k=((b-a)//g*x)%(n//g); out.append(f"{(a+m*k)%l} {l}")
print("\n".join(out))
if __name__=="__main__": main()

```

4 C. Capstone Recurrence

4.1 Restatement

Fast Fibonacci queries modulo $1e9+7$.

4.2 Observations and algorithm

Use the invariant named in the editorial overview. The code processes the input in the natural order and updates exactly the state described by that invariant, so each branch corresponds to one mathematical case rather than an implementation trick.

4.3 Correctness proof

Correctness follows from the invariant proved in the detailed guide for this problem. The code initializes the base state, applies exactly the transition described there, and prints the value represented by the final state. $\square$

4.4 Complexity

The complexity follows from the concrete loops and data structures in the code below; the detailed guide also indicates which optimized structure is needed for larger external versions.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; const long long MOD=1000000007; pair<long long,long long> fib(long long
```

4.6 Full Python solution

```
import sys
MOD=1_000_000_007
def fib(n):
    if n==0: return (0,1)
    a,b=fib(n//2)
    c=a*((2*b-a)%MOD)%MOD
```



```
    d=(a*a+b*b)%MOD
    return (d,(c+d)%MOD) if n%2 else (c,d)
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    print("\n".join(str(fib(x)[0]) for x in data[1:1+data[0]]))
if __name__=="__main__": main()
```

5 What to carry forward

Red-level improvement comes from making the invariant explicit, stress-testing it, and then upsolving the stronger external versions.